

第 6 章

存储结构与磁盘划分

本章讲解了如下内容：

- 一切从 “/” 开始；
- 物理设备的命名规则；
- 文件系统与数据资料；
- 挂载硬件设备；
- 添加硬件设备；
- 添加交换分区；
- 磁盘容量配额；
- 软硬方式链接。

Linux 系统中颇具特色的文件存储结构常常搞得新手头晕脑胀，本章将从 Linux 系统中的文件存储结构开始，讲述文件系统层次化标准（FHS，Filesystem Hierarchy Standard）、udev 硬件命名规则以及硬盘分区的规划方法。

为了让读者更好地理解文件系统的作用，刘遑老师将在本章中详细地分析 Linux 系统中最常见的 Ext3、Ext4 与 XFS 文件系统的不同之处，并带领各位读者着重练习硬盘设备分区、格式化以及挂载等常用的硬盘管理操作，以便熟练掌握文件系统的使用方法。

在打下坚实的理论基础与完成一些相关的实践练习后，我们将进一步完整地部署 SWAP 交换分区、配置 quota 磁盘配额服务，以及掌握 ln 命令带来的软硬链接。相信各位读者在学习完本章后，会对 Linux 系统以及 Windows 系统中的磁盘存储以及文件系统有深入的理解。

6.1 一切从 “/” 开始

在 Linux 系统中，目录、字符设备、块设备、套接字、打印机等都被抽象成了文件，即刘遑老师所一直强调的“Linux 系统中一切都是文件”。既然平时我们打交道的都是文件，那么又应该如何找到它们呢？在 Windows 操作系统中，想要找到一个文件，我们要依次进入该文件所在的磁盘分区（假设这里是 D 盘），然后在进入该分区下的具体目录，最终找到这个文件。但是在 Linux 系统中并不存在 C/D/E/F 等盘符，Linux 系统中的一切文件都是从“根（/）”目录开始的，并按照文件系统层次化标准（FHS）采用树形结构来存放文件，以及定义了常见目录的用途。另外，Linux 系统中的文件和目录名称是严格区分大小写的。例如，root、rOOt、Root、rooT 均代表不同的目录，并且文件名称中不得包含斜杠（/）。Linux 系统中的文件存储结构如图 6-1 所示。

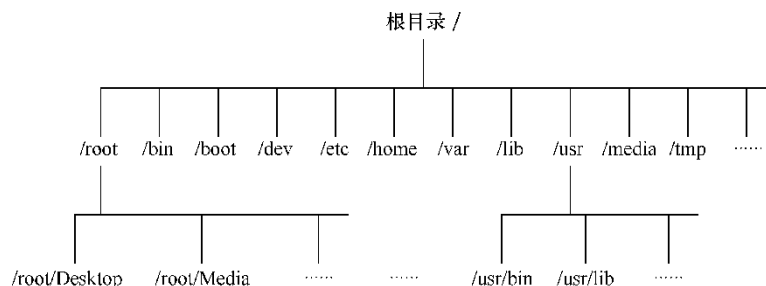


图 6-1 Linux 系统中的文件存储结构

前文提到的 FHS 是根据以往无数 Linux 系统用户和开发者的经验而总结出来的，是用户在 Linux 系统中存储文件时需要遵守的规则，用于指导我们应该把文件保存到什么位置，以及告诉用户应该在何处找到所需的文件。但是，FHS 对于用户来讲只能算是一种道德上的约束，有些用户就是懒得遵守，依然会把文件到处乱放，有些甚至从来没有听说过它。这里并不是号召各位读者去谴责他们，而是建议大家要灵活运用所学的知识，千万不要认准这个 FHS 协定只讲死道理，不然吃亏的可就是自己了。在 Linux 系统中，最常见的目录以及所对应的存放内容如表 6-1 所示。

表 6-1 Linux 系统中常见的目录名称以及相应内容

目录名称	应放置文件的内容
/boot	开机所需文件——内核、开机菜单以及所需配置文件等
/dev	以文件形式存放任何设备与接口
/etc	配置文件
/home	用户家目录
/bin	存放单用户模式下还可以操作的命令
/lib	开机时用到的函数库，以及/bin 与/sbin 下面的命令要调用的函数
/sbin	开机过程中需要的命令
/media	用于挂载设备文件的目录
/opt	放置第三方的软件
/root	系统管理员的家目录
/srv	一些网络服务的数据文件目录
/tmp	任何人都可使用的“共享”临时目录
/proc	虚拟文件系统，例如系统内核、进程、外部设备及网络状态等
/usr/local	用户自行安装的软件
/usr/sbin	Linux 系统开机时不会使用到的软件/命令/脚本
/usr/share	帮助与说明文件，也可放置共享文件
/var	主要存放经常变化的文件，如日志
/lost+found	当文件系统发生错误时，将一些丢失的文件片段存放在这里

在 Linux 系统中另外还有一个重要的概念——路径。路径指的是如何定位到某个文件，分为绝对路径与相对路径。绝对路径指的是从根目录（/）开始写起的文件或目录名称，而相对路径则指的是相对于当前路径的写法。我们来看下面这个例子，以帮助大家理解。假如有

位外国游客来到中国潘家园旅游，当前内急但是找不到洗手间，特意向您问路，那么您有两种正确的指路方法。

- **绝对路径 (absolute path)**:首先坐飞机来到中国，到了北京出首都机场坐机场快轨到三元桥，然后换乘 10 号线到潘家园站，出站后坐 34 路公交车到农光里，下车后路口左转。
- **相对路径 (relative path)**:前面路口左转。

这两种方法都正确。如果您说的是绝对路径，那么任何一位外国游客都可以按照这个提示找到潘家园的洗手间，但是太繁琐了。如果您说的是相对路径，虽然表达很简练，但是这位外国游客只能从当前位置（不见得是潘家园）出发找到洗手间，因此并不能保证在前面的路口左转后可以找到洗手间，由此可见，相对路径不具备普适性。

如果各位读者现在还是不能理解相对路径和绝对路径的区别，也不要着急，以后通过实践练习肯定可以彻底搞明白。当前建议大家先记住 FHS 中规范的目录作用，这将在以后派上用场。

6.2 物理设备的命名规则

在 Linux 系统中一切都是文件，硬件设备也不例外。既然是文件，就必须有文件名称。系统内核中的 udev 设备管理器会自动把硬件名称规范起来，目的是让用户通过设备文件的名字可以猜出设备大致的属性以及分区信息等；这对于陌生的设备来说特别方便。另外，udev 设备管理器的服务会一直以守护进程的形式运行并侦听内核发出的信号来管理/dev 目录下的设备文件。Linux 系统中常见的硬件设备的文件名称如表 6-2 所示。

表 6-2 常见的硬件设备及其文件名称

硬件设备	文件名称
IDE 设备	/dev/hd[a-d]
SCSI/SATA/U 盘	/dev/sd[a-p]
软驱	/dev/fd[0-1]
打印机	/dev/lp[0-15]
光驱	/dev/cdrom
鼠标	/dev/mouse
磁带机	/dev/st0 或 /dev/ht0

由于现在的 IDE 设备已经很少见了，所以一般的硬盘设备都会是以“/dev/sd”开头的。

而一台主机上可以有多块硬盘，因此系统采用 a~p 来代表 16 块不同的硬盘（默认从 a 开始分配），而且硬盘的分区编号也很有讲究：

- 主分区或扩展分区的编号从 1 开始，到 4 结束；
- 逻辑分区从编号 5 开始。

国内很多 Linux 培训讲师以及很多知名 Linux 图书在讲到设备和分区名称时，总会讲错两个知识点。第一个知识点是设备名称的理解错误。很多培训讲师和 Linux 技术图书中会提到，比如/dev/sda 表示主板上第一个插槽上的存储设备，学员或读者在实践操作的时候会发现果然如此，因此也就对这条理论知识更加深信不疑。但真相不是这样的，/dev 目录中 sda 设备之所以是 a，并不是由插槽决定的，而是由系统内核的识别顺序来决定的，而恰巧很多主板的插槽顺序就是系统内核的识别顺序，因此才会被命名为/dev/sda。大家以后在使用 iSCSI 网络存储设备时就会发现，明明主板上第二个插槽是空着的，但系统却能识别到/dev/sdb 这个设备就是这个道理。

第二个知识点是对分区名称的理解错误。很多 Linux 培训讲师会告诉学员，分区的编号代表分区的个数。比如 sda3 表示这是设备上的第三个分区，而学员在做实验的时候确实也会得出这样的结果，但是这个理论知识是错误的，因为分区的数字编码不一定是强制顺延下来的，也有可能是手工指定的。因此 sda3 只能表示是编号为 3 的分区，而不能判断 sda 设备上已经存在了 3 个分区。

在填了这两个“坑”之后，刘遄老师再来分析一下/dev/sda5 这个设备文件名称包含哪些信息，如图 6-2 所示。

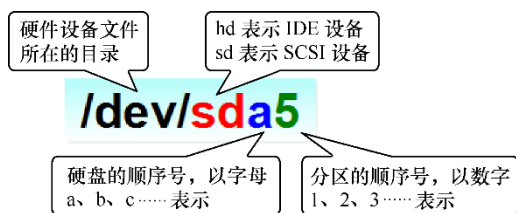


图 6-2 设备文件名称

首先，/dev/目录中保存的应当是硬件设备文件；其次，sd 表示是存储设备；然后，a 表示系统中同类接口中第一个被识别到的设备，最后，5 表示这个设备是一个逻辑分区。一言以蔽之，“/dev/sda5”表示的就是“这是系统中第一块被识别到的硬件设备中分区编号为 5 的逻辑分区的设备文件”。考虑到我们的很多读者完全没有 Linux 基础，不太容易理解前面所说的主分区、扩展分区和逻辑分区的概念，因此接下来简单科普一下硬盘相关的知识。

正是因为计算机有了硬盘设备，我们才可以在玩游戏的过程中或游戏通关之后随时存档，而不用每次重头开始。硬盘设备是由大量的扇区组成的，每个扇区的容量为 512 字节。其中第一个扇区最重要，它里面保存着主引导记录与分区表信息。就第一个扇区来讲，主引导记录需要占用 446 字节，分区表为 64 字节，结束符占用 2 字节；其中分区表中每记录一个分区信息就需要 16 字节，这样一来最多只有 4 个分区信息可以写到第一个扇区中，这 4 个分区就是 4 个主分区。第一个扇区中的数据信息如图 6-3 所示。

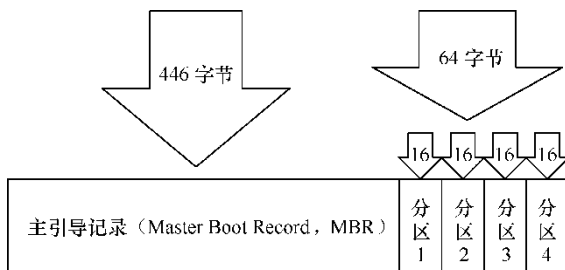


图 6-3 第一个扇区中的数据信息

现在，问题来了——第一个扇区最多只能创建出 4 个分区？于是为了解决分区个数不够的问题，可以将第一个扇区的分区表中 16 字节（原本要写入主分区信息）的空间（称之为扩展分区）拿出来指向另外一个分区。也就是说，扩展分区其实并不是一个真正的分区，而更像是一个占用 16 字节分区表空间的指针——一个指向另外一个分区的指针。这样一来，用户一般会选择使用 3 个主分区加 1 个扩展分区的方法，然后在扩展分区中创建出数个逻辑分区，从而来满足多分区（大于 4 个）的需求。当然，就目前来讲大家只要明白为什么主分区不能超过 4 个就足够了。主分区、扩展分区、逻辑分区可以像图 6-4 那样来规划。

注：

所谓扩展分区，严格地讲它不是一个实际意义的分区，它仅仅是一个指向下一个分区的指针，这种指针结构将形成一个单向链表。

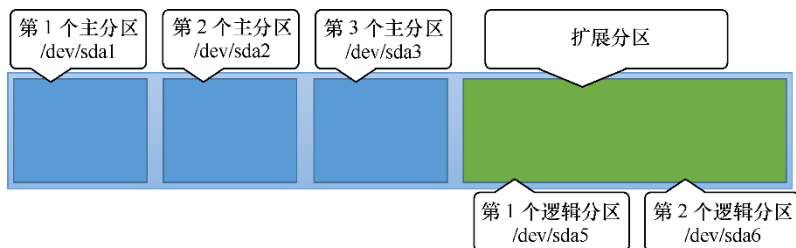


图 6-4 硬盘分区的规划

注：

读者可以试着解读一下/dev/sdb8 的意思。

6.3 文件系统与数据资料

用户在硬件存储设备中执行的文件建立、写入、读取、修改、转存与控制等操作都是依靠文件系统来完成的。文件系统的作用是合理规划硬盘，以保证用户正常的使用需求。Linux 系统支持数十种的文件系统，而最常见的文件系统如下所示。

- **Ext3:** 是一款日志文件系统，能够在系统异常宕机时避免文件系统资料丢失，并能自动修复数据的不一致与错误。然而，当硬盘容量较大时，所需的修复时间也会很

长，而且也不能百分之百地保证资料不会丢失。它会把整个磁盘的每个写入动作的细节都预先记录下来，以便在发生异常宕机后能回溯追踪到被中断的部分，然后尝试进行修复。

- **Ext4**: Ext3 的改进版本，作为 RHEL 6 系统中的默认文件管理系统，它支持的存储容量高达 1EB (1EB=1,073,741,824GB)，且能够有无限多的子目录。另外，Ext4 文件系统能够批量分配 block 块，从而极大地提高了读写效率。
- **XFS**: 是一种高性能的日志文件系统，而且是 RHEL 7 中默认的文件管理系统，它的优势在发生意外宕机后尤其明显，即可以快速地恢复可能被破坏的文件，而且强大的日志功能只用花费极低的计算和存储性能。并且它最大可支持的存储容量为 18EB，这几乎满足了所有需求。

RHEL 7 系统中一个比较大的变化就是使用了 XFS 作为文件系统，这不同于 RHEL 6 使用的 Ext4。从红帽公司官方发布的说明来看，这确实是一个不小的进步，但是刘遑老师在实测中发现并不完全属实。因为单纯就测试一款文件系统的“读取”性能来说，到底要读取多少个文件，每个文件的大小是多少，读取文件时的 CPU、内存等系统资源的占用率如何，以及不同的硬件配置是否会有不同的影响，因此在充分考虑到这些不确定因素后，实在不敢直接照抄红帽官方的介绍。我个人认为 XFS 虽然在性能方面比 Ext4 有所提升，但绝不是压倒性的，因此 XFS 文件系统最卓越的亮点应该当属可支持高达 18EB 的存储容量吧。

就像拿到了一张未裁切的完整纸张那样，我们首先要进行裁切以方便使用，然后在裁切后的纸张上画格以便能书写工整。在拿到了一块新的硬盘存储设备后，也需要先分区，然后再格式化文件系统，最后才能挂载并正常使用。硬盘的分区操作取决于您的需求和硬盘大小；您也可以选择不进行分区，但是必须对硬盘进行格式化处理。接下来刘遑老师再向大家简单地科普一下硬盘在格式化后发生的事情。再次强调，不用刻意去记住，只要能看懂就行了。

日常在硬盘需要保存的数据实在太多了，因此 Linux 系统中有一个名为 super block 的“硬盘地图”。Linux 并不是把文件内容直接写入到这个“硬盘地图”里面，而是在里面记录着整个文件系统的信息。因为如果把所有的文件内容都写入到这里面，它的体积将变得非常大，而且文件内容的查询与写入速度也会变得很慢。Linux 只是把每个文件的权限与属性记录在 inode 中，而且每个文件占用一个独立的 inode 表格，该表格的大小默认为 128 字节，里面记录着如下信息：

- 该文件的访问权限 (read、write、execute)；
- 该文件的所有者与所属组 (owner、group)；
- 该文件的大小 (size)；
- 该文件的创建或内容修改时间 (ctime)；
- 该文件的最后一次访问时间 (atime)；
- 该文件的修改时间 (mtime)；

- 文件的特殊权限（SUID、SGID、SBIT）；
- 该文件的真实数据地址（point）。

而文件的实际内容则保存在 block 块中（大小可以是 1KB、2KB 或 4KB），一个 inode 的默认大小仅为 128B（Ext3），记录一个 block 则消耗 4B。当文件的 inode 被写满后，Linux 系统会自动分配出一个 block 块，专门用于像 inode 那样记录其他 block 块的信息，这样把各个 block 块的内容串到一起，就能够让用户读到完整的文件内容了。对于存储文件内容的 block 块，有下面两种常见情况（以 4KB 的 block 大小为例进行说明）。

- 情况 1：文件很小（1KB），但依然会占用一个 block，因此会潜在地浪费 3KB。
- 情况 2：文件很大（5KB），那么会占用两个 block（5KB-4KB 后剩下的 1KB 也要占用一个 block）。

计算机系统在发展过程中产生了众多的文件系统，为了使用户在读取或写入文件时不用关心底层的硬盘结构，Linux 内核中的软件层为用户程序提供了一个 VFS（Virtual File System，虚拟文件系统）接口，这样用户实际上在操作文件时就是统一对这个虚拟文件系统进行操作了。图 6-5 所示为 VFS 的架构示意图。从中可见，实际文件系统在 VFS 下隐藏了自己的特性和细节，这样用户在日常使用时会觉得“文件系统都是一样的”，也就可以随意使用各种命令在任何文件系统中进行各种操作了（比如使用 `cp` 命令来复制文件）。

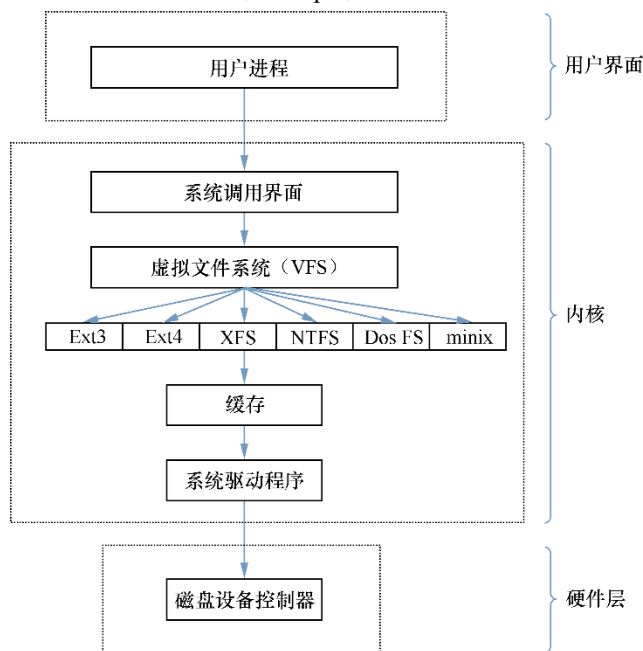


图 6-5 VFS 的架构示意图

6.4 挂载硬件设备

我们在用惯了 Windows 系统后总觉得一切都是理所当然的，平时把 U 盘插入到电脑后也从来没有考虑过 Windows 系统做了哪些事情，才使得我们可以访问这个 U 盘的。接下来我们会逐一学习在 Linux 系统中挂载和卸载存储设备的方法，以便大家更好地了解 Linux 系

统添加硬件设备的工作原理和流程。前面讲到，在拿到一块全新的硬盘存储设备后要先分区，然后格式化，最后才能挂载并正常使用。“分区”和“格式化”大家以前经常听到，但“挂载”又是什么呢？刘遄老师在这里给您一个最简单、最贴切的解释——当用户需要使用硬盘设备或分区中的数据时，需要先将其与一个已存在的目录文件进行关联，而这个关联动作就是“挂载”。下文将向读者逐步讲解如何使用硬盘设备，但是鉴于与挂载相关的理论知识比较复杂，而且很重要，因此决定再拿出一个小节单独讲解，这次希望大家不仅要看懂，而且还要记住。

6.4.1 mount 命令

`mount` 命令用于挂载文件系统，格式为“`mount 文件系统 挂载目录`”。`mount` 命令中可用的参数及作用如表 6-3 所示。挂载是在使用硬件设备前所执行的最后一步操作。只需使用 `mount` 命令把硬盘设备或分区与一个目录文件进行关联，然后就能在这个目录中看到硬件设备中的数据了。对于比较新的 Linux 系统来讲，一般不需要使用 `-t` 参数来指定文件系统的类型，Linux 系统会自动进行判断。而 `mount` 中的 `-a` 参数则厉害了，它会在执行后自动检查 `/etc/fstab` 文件中是否有疏漏被挂载的设备文件，如果有，则进行自动挂载操作。

表 6-3 mount 命令中的参数以及作用

参数	作用
<code>-a</code>	挂载所有在 <code>/etc/fstab</code> 中定义的文件系统
<code>-t</code>	指定文件系统的类型

例如，要把设备 `/dev/sdb2` 挂载到 `/backup` 目录，只需要在 `mount` 命令中填写设备与挂载目录参数就行，系统会自动去判断要挂载文件的类型，因此只需要执行下述命令即可：

```
[root@linuxprobe ~]# mount /dev/sdb2 /backup
```

虽然按照上面的方法执行 `mount` 命令后就能立即使用文件系统了，但系统在重启后挂载就会失效，也就是说我们需要每次开机后都手动挂载一下。这肯定不是我们想要的效果，如果想让硬件设备和目录永久地进行自动关联，就必须把挂载信息按照指定的填写格式“设备文件 挂载目录 格式类型 权限选项 是否备份 是否自检”（各字段的意义见表 6-4）写入到 `/etc/fstab` 文件中。这个文件中包含着挂载所需的诸多信息项目，一旦配置好之后就能一劳永逸了。

表 6-4 用于挂载信息的指定填写格式中，各字段所表示的意义

字段	意义
设备文件	一般为设备的路径+设备名称，也可以写唯一识别码（UUID，Universally Unique Identifier）
挂载目录	指定要挂载到的目录，需在挂载前创建好
格式类型	指定文件系统的格式，比如 Ext3、Ext4、XFS、SWAP、iso9660（此为光盘设备）等
权限选项	若设置为 defaults，则默认权限为：rw, suid, dev, exec, auto, nouser, async
是否备份	若为 1 则开机后使用 <code>dump</code> 进行磁盘备份，为 0 则不备份
是否自检	若为 1 则开机后自动进行磁盘自检，为 0 则不自检

如果想将文件系统为 `ext4` 的硬件设备 `/dev/sdb2` 在开机后自动挂载到 `/backup` 目录上，并保持默认权限且无需开机自检，就需要在 `/etc/fstab` 文件中写入下面的信息，这样在系统重启后也会成功挂载。

```
[root@linuxprobe ~]# vim /etc/fstab
#
# /etc/fstab
# Created by anaconda on Wed May 4 19:26:23 2017
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
/dev/mapper/rhel-root                /                xfs            defaults      1 1
UUID=812b1f7c-8b5b-43da-8c06-b9999e0fe48b /boot            xfs            defaults      1 2
/dev/mapper                        /rhel-swap       swap swap       defaults      0 0
/dev/cdrom                        /media/cdrom     iso9660        defaults      0 0
/dev/sdb2                        /backup          ext4           defaults      0 0
```

6.4.2 umount 命令

`umount` 命令用于撤销已经挂载的设备文件，格式为“`umount [挂载点/设备文件]`”。我们挂载文件系统的目的是为了使用硬件资源，而卸载文件系统就意味着不再使用硬件的设备资源；相对应地，挂载操作就是把硬件设备与目录进行关联的动作，因此卸载操作只需要说明想要取消关联的设备文件或挂载目录的其中一项即可，一般不需要加其他额外的参数。我们来尝试手动卸载掉 `/dev/sdb2` 设备文件：

```
[root@linuxprobe ~]# umount /dev/sdb2
```

6.5 添加硬盘设备

根据前文讲解的与管理硬件设备相关的理论知识，我们先来理清一下添加硬盘设备的操作思路：首先需要在虚拟机中模拟添加一块新的硬盘存储设备，然后再进行分区、格式化、挂载等操作，最后通过检查系统的挂载状态并真实地使用硬盘来验证硬盘设备是否成功添加。

鉴于我们不需要为了做这个实验而特意买一块真实的硬盘，而是通过虚拟机软件进行硬件模拟，因此这再次体现出了使用虚拟机软件的好处。具体的操作步骤如下。

首先把虚拟机系统关机，稍等几分钟会自动返回到虚拟机管理主界面，然后单击“编辑虚拟机设置”选项，在弹出的界面中单击“添加”按钮，新增一块硬件设备，如图 6-6 所示。

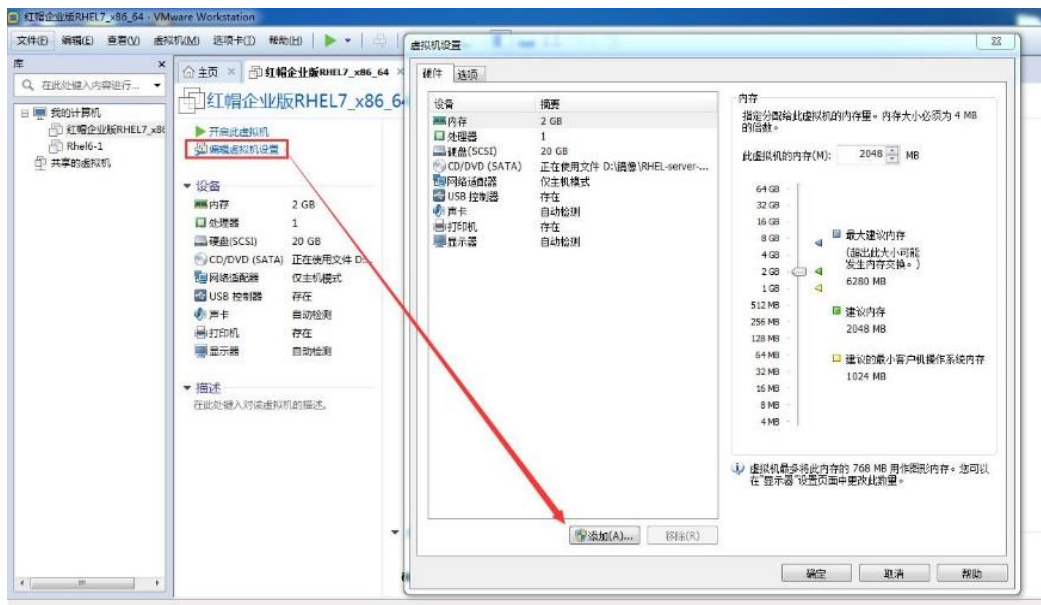


图 6-6 在虚拟机系统中添加硬件设备

选择想要添加的硬件类型为“硬盘”，然后单击“下一步”按钮就可以了，这确实没有什么需要进一步解释的，如图 6-7 所示。

选择虚拟硬盘的类型为 SCSI (默认推荐)，并单击“下一步”按钮，这样虚拟机中的设备名称过一会儿后应该为/dev/sdb，如图 6-8 所示。

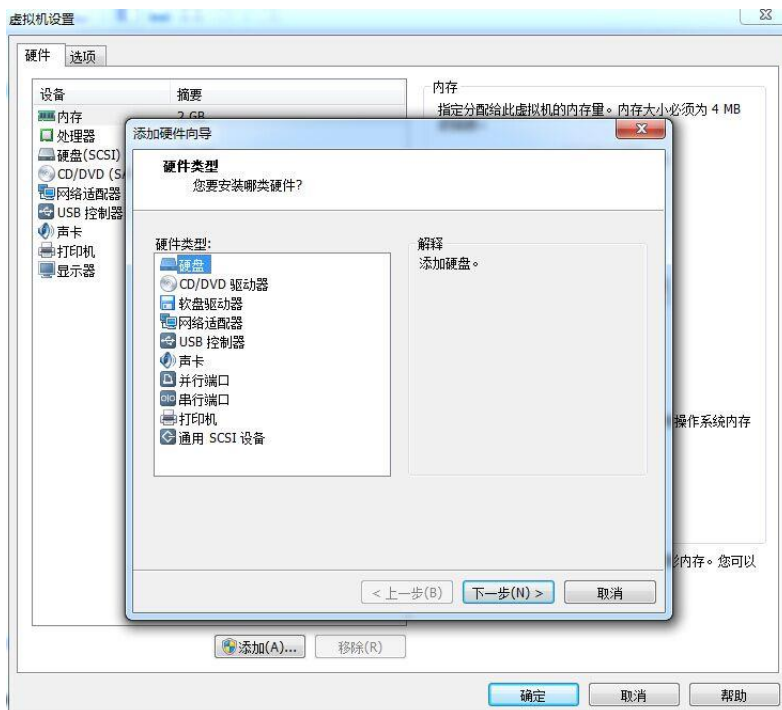


图 6-7 选择添加硬件类

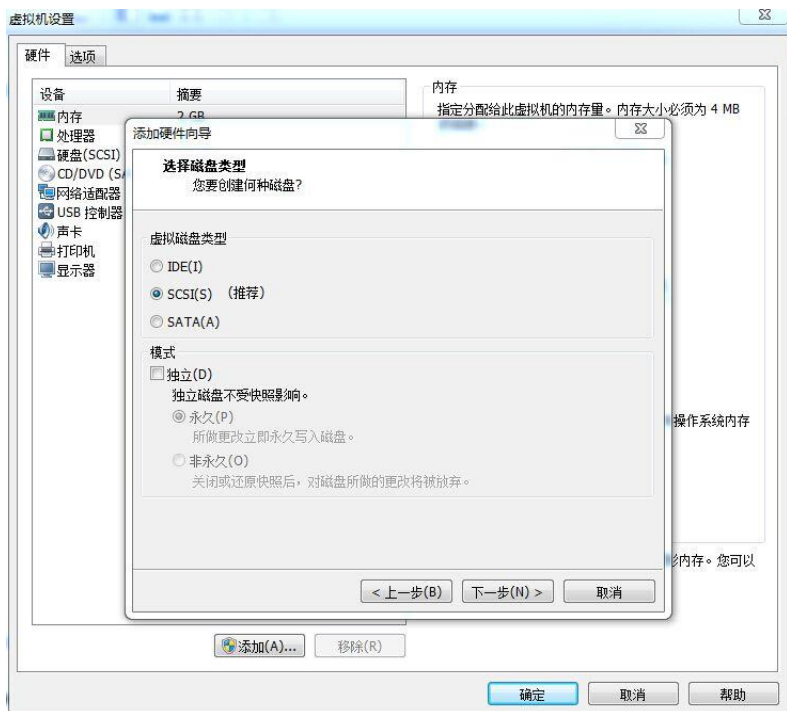


图 6-8 选择硬盘设备类型

选中“创建新虚拟磁盘”单选按钮，而不是其他选项，再次单击“下一步”按钮，如图 6-9 所示。

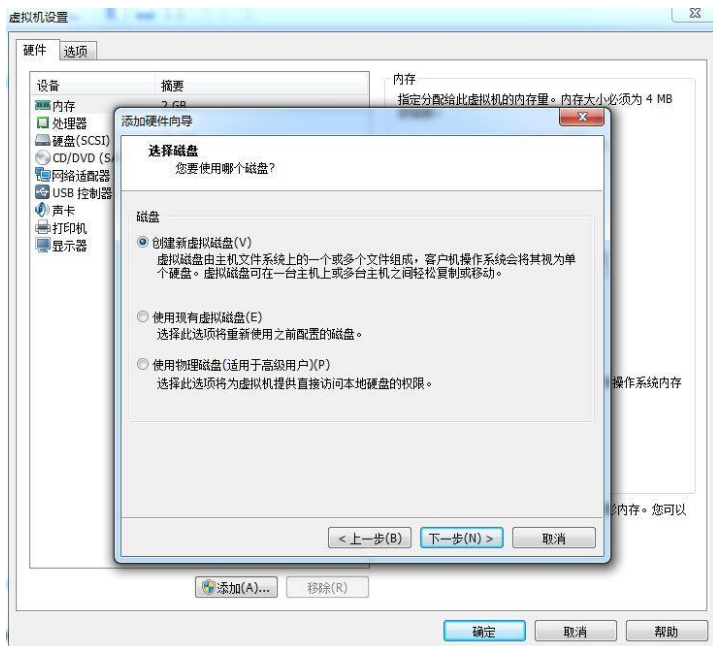


图 6-9 选择“创建新虚拟磁盘”选

将“最大磁盘大小”设置为默认的 20GB。这个数值是限制这台虚拟机所使用的最大硬盘空间，而不是立即将其填满，因此默认 20GB 就很合适了。单击“下一步”按钮，如图 6-10 所示。

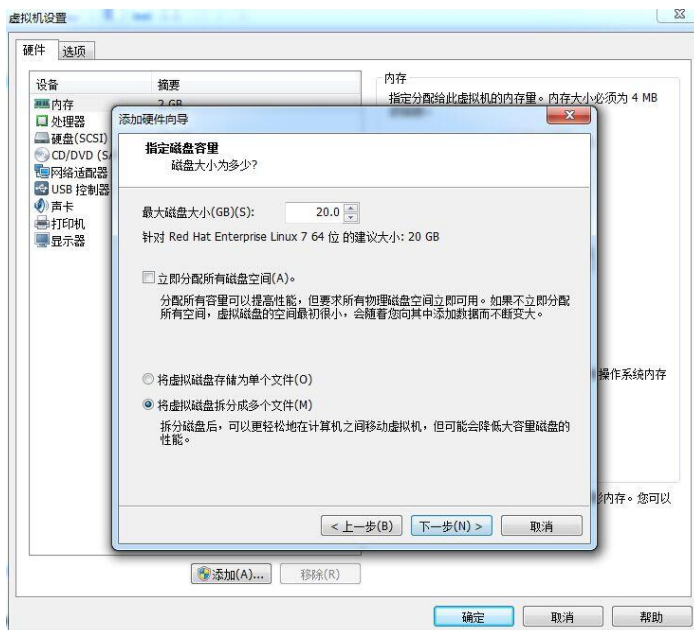


图 6-10 设置硬盘的最大使用空间

设置磁盘文件的文件名和保存位置（这里采用默认设置即可，无需修改），直接单击“完成”按钮，如图 6-11 所示。

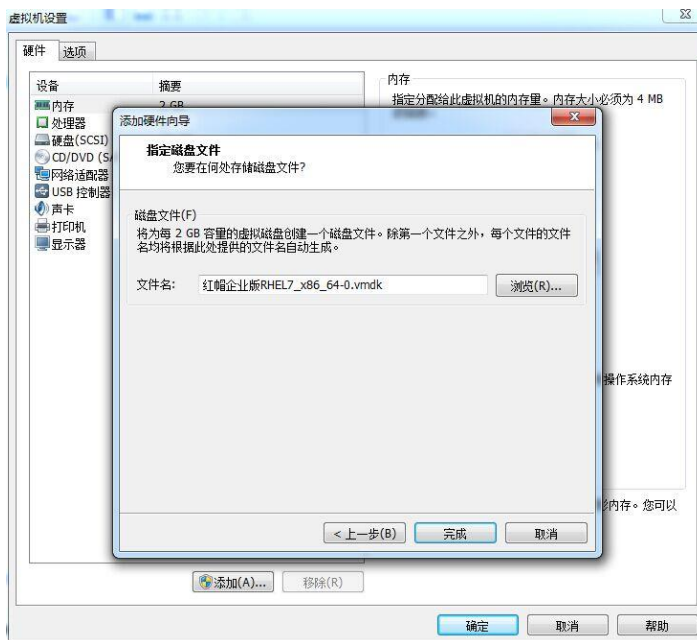


图 6-11 设置磁盘文件的文件名和保存位置

将新硬盘添加好后就可以看到设备信息了。这里不需要做任何修改，直接单击“确认”按钮后就可以开启虚拟机了，如图 6-12 所示。

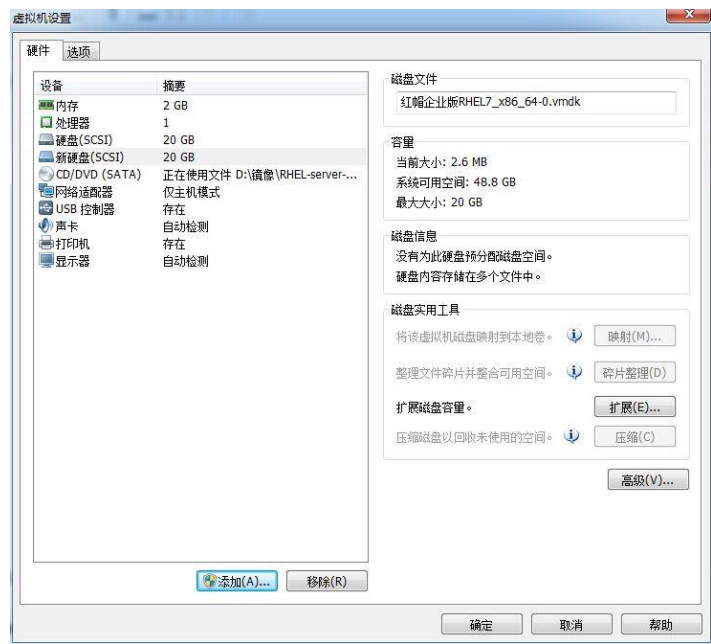


图 6-12 查看虚拟机硬件设置信息

在虚拟机中模拟添加了硬盘设备后就应该能看到抽象成的硬盘设备文件了。按照前文讲解的 udev 服务命名规则，第二个被识别的 SCSI 设备应该会被保存为/dev/sdb，这个就是硬盘设备文件了。但在开始使用该硬盘之前还需要进行分区操作，例如从中取出一个 2GB 的分区设备以供后面的操作使用。

6.5.1 fdisk 命令

在 Linux 系统中，管理硬盘设备最常用的方法就当属 fdisk 命令了。fdisk 命令用于管理磁盘分区，格式为“fdisk [磁盘名称]”，它提供了集添加、删除、转换分区等功能于一身的“一站式分区服务”。不过与前面讲解的直接写到命令后面的参数不同，这条命令的参数（见表 6-5）是交互式的，因此在管理硬盘设备时特别方便，可以根据需求动态调整。

表 6-5 fdisk 命令中的参数以及作用

参数	作用
m	查看全部可用的参数
n	添加新的分区
d	删除某个分区信息
l	列出所有可用的分区类型
t	改变某个分区的类型
p	查看分区信息
w	保存并退出

q	不保存直接退出
---	---------

我们首先使用 `fdisk` 命令来尝试管理 `/dev/sdb` 硬盘设备。在看到提示信息后输入参数 `p` 来查看硬盘设备内已有的分区信息，其中包括了硬盘的容量大小、扇区个数等信息：

```
[root@linuxprobe ~]# fdisk /dev/sdb
Welcome to fdisk (util-linux 2.23.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.
Device does not contain a recognized partition table
Building a new DOS disklabel with disk identifier 0x47d24a34.
Command (m for help): p
Disk /dev/sdb: 21.5 GB, 21474836480 bytes, 41943040 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x47d24a34

Device Boot Start End Blocks Id System
```

输入参数 `n` 尝试添加新的分区。系统会要求您是选择继续输入参数 `p` 来创建主分区，还是输入参数 `e` 来创建扩展分区。这里输入参数 `p` 来创建一个主分区：

```
Command (m for help): n
Partition type:
p primary (0 primary, 0 extended, 4 free)
e extended
Select (default p): p
```

在确认创建一个主分区后，系统要求您先输入主分区的编号。我们在前文得知，主分区的编号范围是 1~4，因此这里输入默认的 1 就可以了。接下来系统会提示定义起始的扇区位置，这不需要改动，我们敲击回车键保留默认设置即可，系统会自动计算出最靠前的空闲扇区的位置。最后，系统会要求定义分区的结束扇区位置，这其实就是要去定义整个分区的大小是多少。我们不用去计算扇区的个数，只需要输入+2G 即可创建出一个容量为 2GB 的硬盘分区。

```
Partition number (1-4, default 1): 1
First sector (2048-41943039, default 2048): 此处敲击回车
Using default value 2048
Last sector, +sectors or +size{K,M,G} (2048-41943039, default 41943039): +2G
Partition 1 of type Linux and of size 2 GiB is set
```

再次使用参数 `p` 来查看硬盘设备中的分区信息。果然就能看到一个名称为 `/dev/sdb1`、起始扇区位置为 2048、结束扇区位置为 4196351 的主分区了。这时候千万不要直接关闭窗口，而应该敲击参数 `w` 后回车，这样分区信息才是真正的写入成功啦。

```
Command (m for help): p
Disk /dev/sdb: 21.5 GB, 21474836480 bytes, 41943040 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0x47d24a34

Device Boot Start End Blocks Id System
```

```
/dev/sdb1 2048 4196351 2097152 83 Linux
Command (m for help): w
The partition table has been altered!
Calling ioctl() to re-read partition table.
Syncing disks.
```

在上述步骤执行完毕之后，Linux 系统会自动把这个硬盘主分区抽象成/dev/sdb1 设备文件。我们可以使用 file 命令查看该文件的属性，但是刘遛老师在讲课和工作中发现，有些时候系统并没有自动把分区信息同步给 Linux 内核，而且这种情况似乎还比较常见（但不能算是严重的 bug）。我们可以输入 partprobe 命令手动将分区信息同步到内核，而且一般推荐连续两次执行该命令，效果会更好。如果使用这个命令都无法解决问题，那么就重启计算机吧，这个杀手锏百试百灵，一定会有用的。

```
[root@linuxprobe ~]# file /dev/sdb1
/dev/sdb1: cannot open (No such file or directory)
[root@linuxprobe ~]# partprobe
[root@linuxprobe ~]# partprobe
[root@linuxprobe ~]# file /dev/sdb1
/dev/sdb1: block special
```

如果硬件存储设备没有进行格式化，则 Linux 系统无法得知怎么在其上写入数据。因此，在对存储设备进行分区后还需要进行格式化操作。在 Linux 系统中用于格式化操作的命令是 mkfs。这条命令很有意思，因为在 Shell 终端中输入 mkfs 名后再敲击两下用于补齐命令的 Tab 键，会有如下所示的效果：

```
[root@linuxprobe ~]# mkfs
mkfs          mkfs.cramfs    mkfs.ext3     mkfs.fat      mkfs.msdos    mkfs.xfs
mkfs.btrfs    mkfs.ext2      mkfs.ext4     mkfs.minix    mkfs.vfat
```

对！这个 mkfs 命令很贴心地把常用的文件系统名称用后缀的方式保存成了多个命令文件，用起来也非常简单——mkfs.文件类型名称。例如要格式分区为 XFS 的文件系统，则命令应为 mkfs.xfs /dev/sdb1。

```
[root@linuxprobe ~]# mkfs.xfs /dev/sdb1
meta-data=/dev/sdb1 isize=256 agcount=4, agsize=131072 blks
= sectsz=512 attr=2, projid32bit=1
= crc=0
data = bsize=4096 blocks=524288, imaxpct=25
= sunit=0 swidth=0 blks
naming =version 2 bsize=4096 ascii-ci=0 ftype=0
log =internal log bsize=4096 blocks=2560, version=2
= sectsz=512 sunit=0 blks, lazy-count=1
realtime =none extsz=4096 blocks=0, rtextents=0
```

终于完成了存储设备的分区和格式化操作，接下来就是要来挂载并使用存储设备了。与之相关的步骤也非常简单：首先是创建一个用于挂载设备的挂载点目录；然后使用 mount 命令将存储设备与挂载点进行关联；最后使用 df -h 命令来查看挂载状态和硬盘使用量信息。

```
[root@linuxprobe ~]# mkdir /newFS
[root@linuxprobe ~]# mount /dev/sdb1 /newFS/
[root@linuxprobe ~]# df -h
Filesystem                Size      Used Avail    Use% Mounted on
```

/dev/mapper/rhel-root	18G	3.5G	15G	20%	/
devtmpfs	905M	0	905M	0%	/dev
tmpfs	914M	140K	914M	1%	/dev/shm
tmpfs	914M	8.8M	905M	1%	/run
tmpfs	914M	0	914M	0%	/sys/fs/cgroup
/dev/sr0	3.5G	3.5G	0	100%	/media/cdrom
/dev/sda1	497M	119M	379M	24%	/boot
/dev/sdb1	2.0G	33M	2.0G	2%	/newFS

6.5.2 du 命令

既然存储设备已经顺利挂载，接下来就可以尝试通过挂载点目录向存储设备中写入文件了。在写入文件之前，先介绍一个用于查看文件数据占用量的 `du` 命令，其格式为“`du [选项] [文件]`”。简单来说，该命令就是用来查看一个或多个文件占用了多大的硬盘空间。我们还可以使用 `du -sh /*` 命令来查看在 Linux 系统根目录下所有一级目录分别占用的空间大小。下面，我们先从某些目录中复制过来一批文件，然后查看这些文件总共占用了多大的容量：

```
[root@linuxprobe ~]# cp -rf /etc/* /newFS/
[root@linuxprobe ~]# ls /newFS/
abrt hosts pulse
adjtime hosts.allow purple
aliases hosts.deny qemu-ga
aliases.db hp qemu-kvm
alsa idmapd.conf radvd.conf
alternatives init.d rc0.d
anacrontab inittab rc1.d
.....省略部分输入信息.....
[root@linuxprobe ~]# du -sh /newFS/
33M /newFS/
```

细心的读者一定还记得，前面在讲解 `mount` 命令时提到，使用 `mount` 命令挂载的设备文件会在系统下一次重启的时候失效。如果想让这个设备文件的挂载永久有效，则需要把挂载的信息写入到配置文件中：

```
[root@linuxprobe ~]# vim /etc/fstab
#
# /etc/fstab
# Created by anaconda on Wed May 4 19:26:23 2017
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
/dev/mapper/rhel-root / xfs defaults 1 1
UUID=812b1f7c-8b5b-43da-8c06-b9999e0fe48b /boot xfs defaults 1 2
/dev/mapper /rhel-swap swap swap defaults 0 0
/dev/cdrom /media/cdrom iso9660 defaults 0 0
/dev/sdb1 /newFS xfs defaults 0 0
```

6.6 添加交换分区

SWAP（交换）分区是一种通过在硬盘中预先划分一定的空间，然后将把内存中暂时不常用的数据临时存放到硬盘中，以便腾出物理内存空间让更活跃的程序服务来使用的技术，其设计目的是为了解决真实物理内存不足的问题。但由于交换分区毕竟是通过硬盘设备读写数据的，速度肯定要比物理内存慢，所以只有当真实的物理内存耗尽后才会调用交换分区的资源。

交换分区的创建过程与前文讲到的挂载并使用存储设备的过程非常相似。在对/dev/sdb 存储设备进行分区操作前，有必要先说一下交换分区的划分建议：在生产环境中，交换分区的大小一般为真实物理内存的 1.5~2 倍，为了让大家更明显地感受交换分区空间的变化，这里取出一个大小为 5GB 的主分区作为交换分区资源。在分区创建完毕后保存并退出即可：

```
[root@linuxprobe ~]# fdisk /dev/sdb
Welcome to fdisk (util-linux 2.23.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.
Device does not contain a recognized partition table
Building a new DOS disklabel with disk identifier 0xb3d27ce1.
Command (m for help): n
Partition type:
p primary (1 primary, 0 extended, 3 free)
e extendedSelect (default p): p
Partition number (2-4, default 2):
First sector (4196352-41943039, default 4196352): 此处敲击回车
Using default value 4196352
Last sector, +sectors or +size{K,M,G} (4196352-41943039, default 41943039): +5G
Partition 2 of type Linux and of size 5 GiB is set
Command (m for help): p
Disk /dev/sdb: 21.5 GB, 21474836480 bytes, 41943040 sectors
Units = sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disk label type: dos
Disk identifier: 0xb0ced57f

   Device Boot      Start         End      Blocks    Id  System
/dev/sdb1            2048        4196351    2097152     83   Linux
/dev/sdb2    4196352    14682111    5242880     83   Linux
Command (m for help): w
The partition table has been altered!
Calling ioctl() to re-read partition table.
WARNING: Re-reading the partition table failed with error 16: Device or resource busy.
The kernel still uses the old table. The new table will be used at
the next reboot or after you run partprobe(8) or kpartx(8)
Syncing disks.
```

使用 SWAP 分区专用的格式化命令 **mkswap**，对新建的主分区进行格式化操作：

```
[root@linuxprobe ~]# mkswap /dev/sdb2
Setting up swapspace version 1, size = 5242876 KiB
no label, UUID=2972f9cb-17f0-4113-84c6-c64b97c40c75
```

使用 **swapon** 命令把准备好的 SWAP 分区设备正式挂载到系统中。我们可以使用 **free -m** 命令查看交换分区的大小变化（由 2047MB 增加到 7167MB）：

```
[root@linuxprobe ~]# free -m
              total        used         free      shared    buffers     cached
Mem:           1483         782          701           9           0         254
```

```
-/+ buffers/cache:      526      957
Swap:      2047      0      2047
[root@linuxprobe ~]# swapon /dev/sdb2
[root@linuxprobe ~]# free -m
              total        used         free      shared    buffers     cached
Mem:          1483         785          697           9           0         254
-/+ buffers/cache:      530          953
Swap:         7167           0         7167
```

为了能够让新的交换分区设备在重启后依然生效，需要按照下面的格式将相关信息写入到配置文件中，并记得保存：

```
[root@linuxprobe ~]# vim /etc/fstab
#
# /etc/fstab
# Created by anaconda on Wed May 4 19:26:23 2017
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
/dev/mapper/rhel-root / xfs defaults 1 1
UUID=812b1f7c-8b5b-43da-8c06-b9999e0fe48b /boot xfs defaults 1 2
/dev/mapper /rhel-swap swap swap defaults 0 0
/dev/cdrom /media/cdrom iso9660 defaults 0 0
/dev/sdb1 /newFS xfs defaults 0 0
/dev/sdb2 swap swap defaults 0 0
```

6.7 磁盘容量配额

本书在前面曾经讲到，Linux 系统的设计初衷就是让许多人一起使用并执行各自的任務，从而成为多用户、多任务的操作系统。但是，硬件资源是固定且有限的，如果某些用户不断地在 Linux 系统上创建文件或者存放电影，硬盘空间总有一天会被占满。针对这种情况，root 管理员就需要使用磁盘容量配额服务来限制某位用户或某个用户组针对特定文件夹可以使用的最大硬盘空间或最大文件个数，一旦达到这个最大值就不再允许继续使用。可以使用 `quota` 命令进行磁盘容量配额管理，从而限制用户的硬盘可用容量或所能创建的最大文件个数。`quota` 命令还有软限制和硬限制的功能。

- 软限制：当达到软限制时会提示用户，但仍允许用户在限定的额度内继续使用。
- 硬限制：当达到硬限制时会提示用户，且强制终止用户的操作。

RHEL 7 系统中已经安装了 `quota` 磁盘容量配额服务程序包，但存储设备却默认没有开启对 `quota` 的支持，此时需要手动编辑配置文件，让 RHEL 7 系统中的 `/boot` 目录能够支持 `quota` 磁盘配额技术。另外，对于学习过早期的 Linux 系统，或者具有 RHEL 6 系统使用经验的读者来说，这里需要特别注意。早期的 Linux 系统要想让硬盘设备支持 `quota` 磁盘容量配额服务，使用的是 `usrquota` 参数，而 RHEL 7 系统使用的则是 `uquota` 参数。在重启系统后使用 `mount` 命令查看，即可发现 `/boot` 目录已经支持 `quota` 磁盘配额技术了：

```
[root@linuxprobe ~]# vim /etc/fstab
#
# /etc/fstab
# Created by anaconda on Wed May 4 19:26:23 2017
```

```
#
# Accessible filesystems, by reference, are maintained under '/dev/disk'
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info
#
/dev/mapper/rhel-root                /                xfs          defaults    1 1
UUID=812b1f7c-8b5b-43da-8c06-b9999e0fe48b/boot    xfs          defaults,usrquota 1 2
/dev/mapper                        /rhel-swap      swap swap    defaults    0 0
/dev/cdrom                        /media/cdrom    iso9660     defaults    0 0
/dev/sdb1                        /newFS          xfs          defaults    0 0
/dev/sdb2                        swap            swap         defaults    0 0
[root@linuxprobe ~]# reboot
[root@linuxprobe ~]# mount | grep boot
/dev/sda1 on /boot type xfs (rw,relatime,seclabel,attr2,inode64,usrquota)
```

接下来创建一个用于检查 quota 磁盘容量配额效果的用户 tom，并针对/boot 目录增加其他人的写权限，保证用户能够正常写入数据：

```
[root@linuxprobe ~]# useradd tom
[root@linuxprobe ~]# chmod -Rf o+w /boot
```

6.7.1 xfs_quota 命令

xfs_quota 命令是一个专门针对 XFS 文件系统来管理 quota 磁盘容量配额服务而设计的命令，格式为“xfs_quota [参数] 配额 文件系统”。其中，-c 参数用于以参数的形式设置要执行的命令；-x 参数是专家模式，让运维人员能够对 quota 服务进行更多复杂的配置。接下来我们使用 xfs_quota 命令来设置用户 tom 对/boot 目录的 quota 磁盘容量配额。具体的限额控制包括：硬盘使用量的软限制和硬限制分别为 3MB 和 6MB；创建文件数量的软限制和硬限制分别为 3 个和 6 个。

```
[root@linuxprobe ~]# xfs_quota -x -c 'limit bsoft=3m bhard=6m isoft=3 ihard=6
tom' /boot
[root@linuxprobe ~]# xfs_quota -x -c report /boot
User quota on /boot (/dev/sda1)  Blocks
User ID Used Soft Hard Warn/Grace
-----
root 95084    0    0    00 [-----]
tom  0       3072 6144 00 [-----]
```

当配置好上述的各种软硬限制后，尝试切换到这个普通用户，然后分别尝试创建一个体积为 5MB 和 8MB 的文件。可以发现，在创建 8MB 的文件时受到了系统限制：

```
[root@linuxprobe ~]# su - tom
[tom@linuxprobe ~]$ dd if=/dev/zero of=/boot/tom bs=5M count=1
1+0 records in
1+0 records out
5242880 bytes (5.2 MB) copied, 0.123966 s, 42.3 MB/s
[tom@linuxprobe ~]$ dd if=/dev/zero of=/boot/tom bs=8M count=1
dd: error writing '/boot/tom': Disk quota exceeded
1+0 records in
0+0 records out
6291456 bytes (6.3 MB) copied, 0.0201593 s, 312 MB/s
```

6.7.2 edquota 命令

`edquota` 命令用于编辑用户的 `quota` 配额限制，格式为“`edquota [参数] [用户]`”。在为用户设置了 `quota` 磁盘容量配额限制后，可以使用 `edquota` 命令按需修改限额的数值。其中，`-u` 参数表示要针对哪个用户进行设置；`-g` 参数表示要针对哪个用户组进行设置。`edquota` 命令会调用 `Vi` 或 `Vim` 编辑器来让 `root` 管理员修改要限制的具体细节。下面把用户 `tom` 的硬盘使用量的硬限额从 `5MB` 提升到 `8MB`：

```
[root@linuxprobe ~]# edquota -u tom
Disk quotas for user tom (uid 1001):
Filesystem blocks soft hard inodes soft hard
/dev/sda 6144 3072 8192 1 3 6
[root@linuxprobe ~]# su - tom
Last login: Mon Sep 7 16:43:12 CST 2017 on pts/0
[tom@linuxprobe ~]$ dd if=/dev/zero of=/boot/tom bs=8M count=1
1+0 records in
1+0 records out
8388608 bytes (8.4 MB) copied, 0.0268044 s, 313 MB/s
[tom@linuxprobe ~]$ dd if=/dev/zero of=/boot/tom bs=10M count=1
dd: error writing '/boot/tom': Disk quota exceeded
1+0 records in
0+0 records out
8388608 bytes (8.4 MB) copied, 0.167529 s, 50.1 MB/s
```

6.8 软硬方式链接

当引领大家学习完本章所有的硬盘管理知识之后，刘遄老师终于可以放心大胆地讲解 Linux 系统中的“快捷方式”了。在 Windows 系统中，快捷方式就是指向原始文件的一个链接文件，可以让用户从不同的位置来访问原始的文件；原文件一旦被删除或剪切到其他地方后，会导致链接文件失效。但是，这个看似简单的东西在 Linux 系统中可不太一样。

在 Linux 系统中存在硬链接和软链接两种文件。

- **硬链接 (hard link)：**可以将它理解为一个“指向原始文件 `inode` 的指针”，系统不为它分配独立的 `inode` 和文件。所以，硬链接文件与原始文件其实是同一个文件，只是名字不同。我们每添加一个硬链接，该文件的 `inode` 连接数就会增加 1；而且只有当该文件的 `inode` 连接数为 0 时，才算彻底将它删除。换言之，由于硬链接实际上是指向原文件 `inode` 的指针，因此即便原始文件被删除，依然可以通过硬链接文件来访问。需要注意的是，由于技术的局限性，我们不能跨分区对目录文件进行链接。
- **软链接 (也称为符号链接[symbolic link])：**仅仅包含所链接文件的路径名，因此能链接目录文件，也可以跨越文件系统进行链接。但是，当原始文件被删除后，链接文件也将失效，从这一点上来说与 Windows 系统中的“快捷方式”具有一样的性质。

ln 命令

ln 命令用于创建链接文件，格式为“ln [选项] 目标”，其可用的参数以及作用如表 6-6 所示。在使用 ln 命令时，是否添加-s 参数，将创建出性质不同的两种“快捷方式”。因此如果没有扎实的理论知识和实践经验做铺垫，尽管能够成功完成实验，但永远不会明白为什么会成功。

表 6-6 ln 命令中可用的参数以及作用

参数	作用
-s	创建“符号链接”（如果不带-s 参数，则默认创建硬链接）
-f	强制创建文件或目录的链接
-i	覆盖前先询问
-v	显示创建链接的过程

为了更好地理解软链接、硬链接的不同性质，接下来创建一个类似于 Windows 系统中快捷方式的软链接。这样，当原始文件被删除后，就无法读取新建的链接文件了。

```
[root@linuxprobe ~]# echo "Welcome to linuxprobe.com" > readme.txt
[root@linuxprobe ~]# ln -s readme.txt readit.txt
[root@linuxprobe ~]# cat readme.txt
Welcome to linuxprobe.com
[root@linuxprobe ~]# cat readit.txt
Welcome to linuxprobe.com
[root@linuxprobe ~]# ls -l readme.txt
-rw-r--r-- 1 root root 26 Jan 11 00:08 readme.txt
[root@linuxprobe ~]# rm -f readme.txt
[root@linuxprobe ~]# cat readit.txt
cat: readit.txt: No such file or directory
```

接下来针对一个原始文件创建一个硬链接，即相当于针对原始文件的硬盘存储位置创建了一个指针，这样一来，新创建的这个硬链接就不再依赖于原始文件的名称等信息，也不会因为原始文件的删除而导致无法读取。同时可以看到创建硬链接后，原始文件的硬盘链接数量增加到了 2。

```
[root@linuxprobe ~]# echo "Welcome to linuxprobe.com" > readme.txt
[root@linuxprobe ~]# ln readme.txt readit.txt
[root@linuxprobe ~]# cat readme.txt
Welcome to linuxprobe.com
[root@linuxprobe ~]# cat readit.txt
Welcome to linuxprobe.com
[root@linuxprobe ~]# ls -l readme.txt
-rw-r--r-- 2 root root 26 Jan 11 00:13 readme.txt
[root@linuxprobe ~]# rm -f readme.txt
[root@linuxprobe ~]# cat readit.txt
Welcome to linuxprobe.com
```

复习题

1. /home 目录与/root 目录内存放的文件有何相同点以及不同点？

答：这两个目录都是用来存放用户的家目录数据的，但是，/root 目录存放的是 root 管理员的家目录数据。

2. 假如一个设备的文件名称为/dev/sdb，可以确认它是主板第二个插槽上的设备吗？

答：不一定，因为设备的文件名称是由系统的识别顺序来决定的。

3. 如果硬盘中需要 5 个分区，至少需要几个逻辑分区？

答：可以选用创建 3 个主分区+1 个扩展分区的方法，然后把扩展分区再分成 2 个逻辑分区，即有了 5 个分区。

4. /dev/sda5 是主分区还是逻辑分区？

答：逻辑分区。

5. 哪个服务决定了设备在/dev 目录中的名称？

答：udev 设备管理器服务。

6. 用一句话来描述挂载操作。

答：当用户需要使用硬盘设备或分区中的数据时，需要先将其与一个已存在的目录文件进行关联，而这个关联动作就是“挂载”。

7. 在配置 quota 磁盘容量配额服务时，软限制数值必须小于硬限制数值么？

答：不一定，软限制数值可以小于等于硬限制数值。

8. 若原始文件被改名，那么之前创建的硬链接还能访问到这个原始文件么？

答：可以。